

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1714

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration : 1 Hour

Total Marks:50

Annexure A

Design Task

Code of Conduct:

I S.PRAVEEN (192524239) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Distributed Voice Assistant for Smart Devices

Design of a distributed AI voice-assistant system using PySpark and RDD operations to process speech streams from millions of smart devices, with a multi-agent layer for personalization, multi-device context, and continuous accuracy improvement.

1. Introduction

Millions of smart devices — phones, speakers, wearables, vehicles, and smart-home hubs — continuously generate voice queries that must be converted to text, understood, and answered within a second or two. A single server cannot process this volume of concurrent audio streams. This design uses **Apache Spark (PySpark)**, whose Resilient Distributed Dataset (RDD) model was built exactly for this kind of large-scale, fault-tolerant, parallel data processing, combined with a layer of cooperating **intelligent agents** that add personalization, context-awareness, and learning on top of the raw pipeline.

2. System Architecture

Audio from devices first lands in a streaming ingestion layer, then flows through a chain of RDD transformations inside the PySpark cluster, and finally reaches a multi-agent intelligence layer that produces and personalizes the response, as shown below.

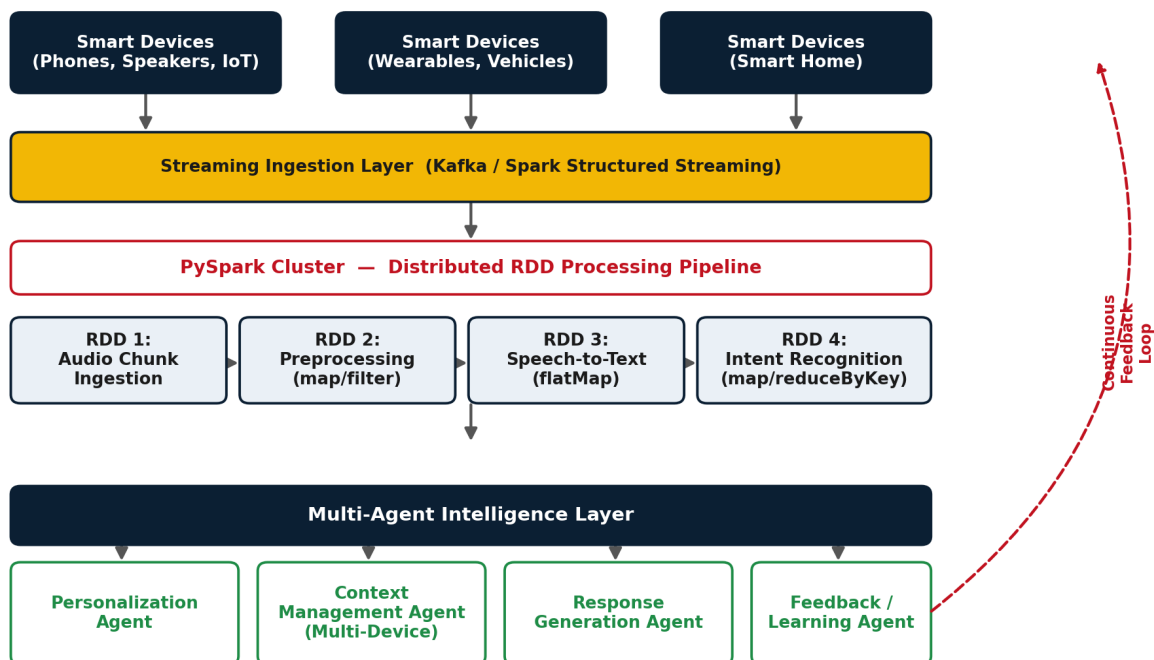


Figure 1: Distributed architecture of the voice assistant pipeline

3. RDD Transformations for the Speech-to-Text & Intent Pipeline

Each incoming audio chunk, tagged with a device ID and timestamp, becomes a record inside an RDD. The pipeline applies a sequence of RDD transformations — all of which are lazy and only execute when an action is triggered, letting Spark optimize and parallelize the whole chain across

the cluster:

- **Ingestion RDD:** Raw audio chunks from Kafka/Spark Structured Streaming are loaded into an RDD, partitioned automatically across worker nodes so that no single machine bears the full load.
- **map() — Preprocessing:** Each partition applies noise reduction, silence trimming, and audio normalization in parallel, transforming raw audio bytes into cleaned feature vectors.
- **filter():** Discards empty, corrupted, or below-threshold-confidence audio segments before they consume further compute, keeping the pipeline efficient at scale.
- **flatMap() — Speech-to-Text:** Each audio feature vector is expanded into a sequence of recognized words/tokens by an acoustic model run as a distributed UDF, producing a new RDD of transcribed text keyed by device ID.
- **map() — Intent Extraction:** Transcribed text is mapped to an (intent, entity, device_id) tuple using an NLP intent-classification model applied in parallel across partitions.
- **reduceByKey() / groupByKey():** Aggregates repeated or related intents per user/session (e.g. multiple devices reporting the same household command) so a single, de-duplicated instruction is produced.
- **persist()/cache():** Frequently reused RDDs — such as a user's recent utterance history — are cached in memory across the cluster so repeated agent lookups avoid recomputation.

Because RDDs are partitioned across nodes and track lineage rather than replicating data, the pipeline scales horizontally (more devices → more partitions/executors) and recovers automatically: if a worker node fails mid-stream, Spark recomputes only the lost partition from lineage instead of restarting the entire job — essential when millions of live voice streams cannot be dropped.

4. Multi-Agent Interaction Layer

Once intents are extracted, four cooperating agents take over to turn a raw intent into a personalized, context-aware response:

- **Personalization Agent:** Reads the user's profile and interaction history (cached RDD/feature store) to tailor responses — preferred music service, accent, tone, or frequently asked routines — so the same intent produces a response suited to that specific user.
- **Context Management Agent:** Tracks state across a user's multiple devices (phone, speaker, car). If a query starts on a smart speaker and continues on a phone, this agent maintains a shared session/context object so the assistant remembers what was being discussed, not just the last utterance.
- **Response Generation Agent:** Combines the recognized intent, personalization data, and current context to compose the actual reply or device action, then routes it back to the originating (or most appropriate) device.
- **Feedback / Learning Agent:** Monitors corrections, repeated queries, and explicit user feedback (e.g. "that's wrong", retries, cancellations) and feeds this signal back into the model-retraining RDD pipeline, continuously improving speech and intent recognition accuracy over time.

These agents communicate asynchronously through a shared distributed state store (e.g. a key-value store keyed by user ID), which lets each agent scale and be deployed independently while still acting on a consistent, up-to-date view of the user.

5. Multi-Device Context Management

Every device event carries a common user/household identifier. The Context Management Agent maintains a per-user context object (recent intents, active session, device currently in focus) in a fast in-memory store. RDD jobs update this object as new intents arrive, and every agent reads from the same object — so a command given to a speaker can be completed on a phone, and a conversation is not lost when the user switches devices mid-task.

6. Continuous Accuracy Improvement

User interactions are not discarded after a response is sent. Correction signals, low-confidence transcriptions, and repeated/failed queries are collected into a training RDD. A periodic Spark batch job retrains or fine-tunes the speech-to-text and intent-classification models on this accumulated data, and the updated model is redistributed to the streaming pipeline — creating a continuous feedback loop (shown as the dashed path in Figure 1) that improves recognition accuracy the more the system is used.

7. Conclusion

By combining PySpark's distributed RDD processing for the heavy, parallelizable speech-to-text and intent-recognition workload with a lightweight layer of cooperating agents for personalization, multi-device context, and learning, the system meets the scale requirements of millions of concurrent smart-device streams while still feeling personal and consistent to each individual user.